

HW2 — API Server for Vertex AI Search

Overview

HW1 built a dashboard over a fixed, hand-written dataset. Now you'll build the piece that actually *produces* that data: a FastAPI service that searches the web for job postings using **Vertex AI Search** (Google Cloud's managed search product) and saves the results to **Google Cloud Storage (GCS)**. Your HW1 dashboard should be updated to read files with same prefix from GCS instead of a static file.

Learning Objectives

- Build a FastAPI service: Pydantic request models, a **POST** endpoint, returning structured JSON.
- Call a real Google Cloud API (Vertex AI Search / Discovery Engine) with a service account, and handle its response.
- Write/read objects in Google Cloud Storage.

Prerequisites (one-time GCP setup)

You'll need, in a GCP project you control:

1. A **Vertex AI Search** app/engine configured as a website search engine (covered in the recorded video).
2. A **GCS bucket** to store search results in.
3. A **service account key** (JSON file) with permission to call Discovery Engine and read/write your bucket.
4. A **.env** file (not committed to git) with:

```
GCP_PROJECT_ID=your-project-id
VERTEX_AI_PROJECT_ID=your-project-number
SEARCH_ENGINE_ID=your-search-engine-id
GCP_BUCKET_NAME=your-bucket-name
GCP_SERVICE_ACCOUNT_KEY=/path/to/key.json
```

Background: how the search works

Vertex AI Search's **SearchRequest** has **no dedicated "restrict to these websites" field** — despite what you might expect. The way to restrict results to specific domains is to fold **site:** operators directly into the query string itself, the same way you'd type **site:openai.com** into a Google Search box:

```
query = f"{job_title} (site:google.com OR site:openai.com)"
```

The raw response is a stream of protobuf messages, not plain dicts — convert each one with **MessageToDict(result._pb)** before you try to treat it as JSON.

Provided Files

- `fastapi/user_definition.py` — GCP config loaded from `.env`, and `file_name_prefix`
- `streamlit/user_definition.py` — same idea, for the dashboard side.
- `call_fast_api.py` — a script that `POST`s a search request to your running API. This is what actually triggers a search + GCS save; run it before starting the Streamlit app.

Files You Must Create / Complete

`fastapi/extract_save_data.py` (stub provided)

Three pieces, in dependency order:

1. `parse_google_search_results(search_results, job_list)` — for each `result` from `search_results`, extract `title/link` from `result["document"]["derivedStructData"]`, and the snippet's text. Snippets often contain phrases like *"posted 3 days ago"* — extract that number with a regex (`r"(\d+)\s*days?\s*ago"`) and subtract it from today's date to get an actual `date`. If there's no such phrase, just use today's date. Append `{"title", "link", "snippet", "date"}` dicts to `job_list`. This function doesn't return anything and change `job_list` in place.
2. `call_google_search(search_param)` — build a `discoveryengine.SearchServiceClient` (with `service_account.Credentials.from_service_account_file(...)`), build the `site:-restricted` query described above, call `client.search(request)`, convert results with `MessageToDict`, and run them through `parse_google_search_results`. Wrap the whole thing in `try/except`. On failure, return `JSONResponse(status_code=500, content={"message": ...})` instead of raising a generic error. Hint : **Callers of this function must check `isinstance(result, JSONResponse)` before treating the return value as data** — that distinction matters a lot once you start chaining calls together in later homeworks.
3. `save_to_gcs(gcs_upload_param)` — get a `storage.Client`, get the bucket, get (or create) a blob and upload the data.
4. `search_and_save_jobs` (the `@app.post("/search_and_save/jobs")` endpoint) — wire the above together: build a `GoogleSearch` from the incoming `SearchModel` plus your `.env`-sourced config, call `call_google_search`, bail out early (return it directly) if you get a `JSONResponse` back, otherwise `json.dumps` the result and `save_to_gcs` it at `f'{file_name_prefix}/{today}.json'`.

`streamlit/hw2.py` (stub provided)

- `retrieve_data_from_gcs(...)` — list every json blob (file name should end with `.json`) under `file_name_prefix` in the bucket, parse blob, and combine their `"results"` lists, `"job_titles"`, and `"company_dict"`s into one dictionary (same three keys, same shapes, as HW1's sample data — this is intentional, so the rest of `hw2.py` barely changes from HW1).
- The `if __name__ == '__main__':` block: reuse your HW1 sidebar-checkbox filtering logic verbatim, just sourcing `company_dictionary` from the GCS data instead of a hardcoded dict.

How to Run

How to Run a FastAPI server

```
cd fastapi
fastapi run extract_save_data.py
```

Trigger a search (from your host, in another terminal)

```
python call_fast_api.py
```

Start the Streamlit dashboard

```
cd streamlit
streamlit run hw2.py
```

Visit <http://localhost:8501> — you should see the same kind of dashboard as HW1, now populated from a real Vertex AI Search call.

Provided Tests

`fastapi/test_hw2_api.py` and `streamlit/test_hw2_streamlit.py` are provided as self-checks. These are **live integration tests**. They make real Vertex AI Search calls and read/write your real GCS bucket using the credentials in your `.env`.

```
cd fastapi
fastapi run extract_save_data.py
pytest test_hw2_api.py
cd ../streamlit
pytest test_hw2_streamlit.py # run test_hw2_api.py first
```

`test_hw2_streamlit.py` reads whatever's actually in your GCS bucket, so it depends on `test_hw2_api.py` (or a manual `call_fast_api.py` run) having saved at least one blob first. Try it on multiple days to ensure it can read multiple files and display.

Submission Checklist

- ☐ Do not add additional libraries/packages other than what we have installed in the class virtual environment.
- ☐ `POST /search_and_save/jobs` returns a success message and creates a new blob in your GCS bucket, where the filename is `yyyy_mm_dd.json`.
- ☐ A failed search (e.g. bad `SEARCH_ENGINE_ID`) returns a `500` with a useful `message`, not an unhandled exception.

- ☐ The Streamlit dashboard reads from GCS and renders correctly. See <https://hw2-api-server-555299671262.us-central1.run.app>
- ☐ Ensure all submitted codes pass `pycodestyle` with fewer than 5 issues.

```
pycodestyle hw2.py # should not return more than 5 lines
pycodestyle extract_save_data.py # should not return more than 5 lines
```

- ☐ Ensure that all docstrings on `hw2.py` are deleted and not displayed on the web app.
- ☐ Submit a zip file called (HW2) including **only 2 .py file**, `extract_save_data.py` and `hw2.py`.

References

- [Generative AI App Builder](#)